MoveIt2 Simulation-Reality Linkage

MoveIt2 Simulation-Reality Linkage

1. Course Content
2. Preparation
3. Node Communication
4. Core Source Code Analysis

1. Course Content

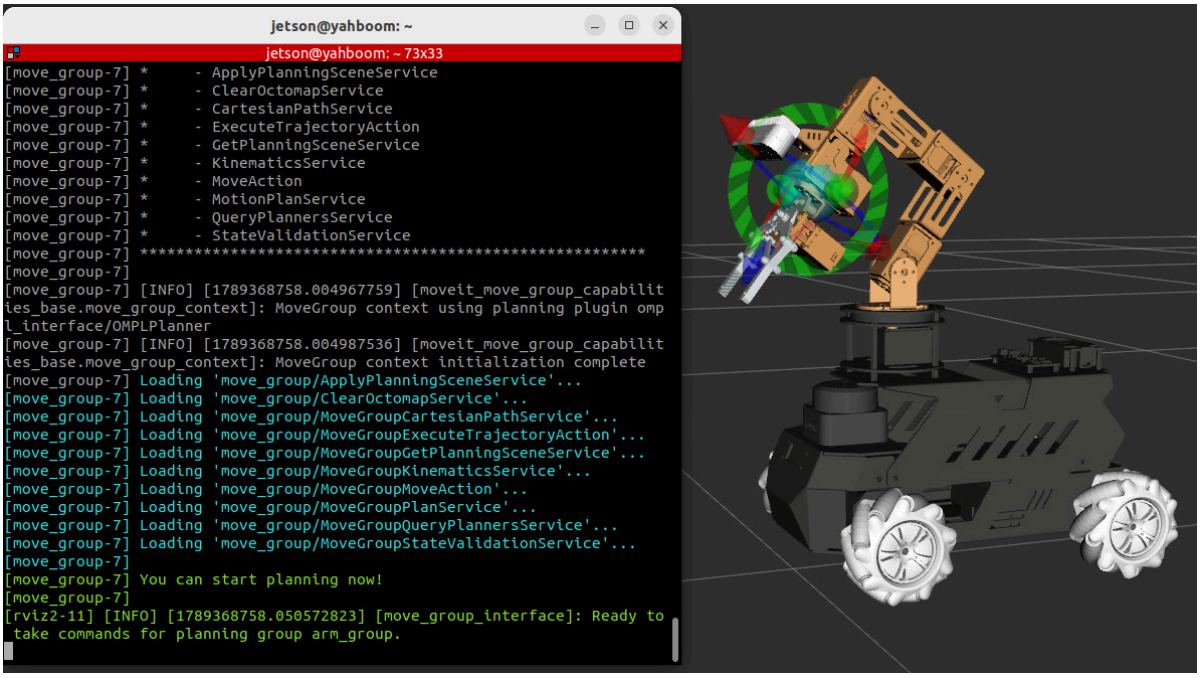
This section explains how to combine the simulated robot arm in RViz with the real robot arm, so as to drive the real machine.

2. Preparation

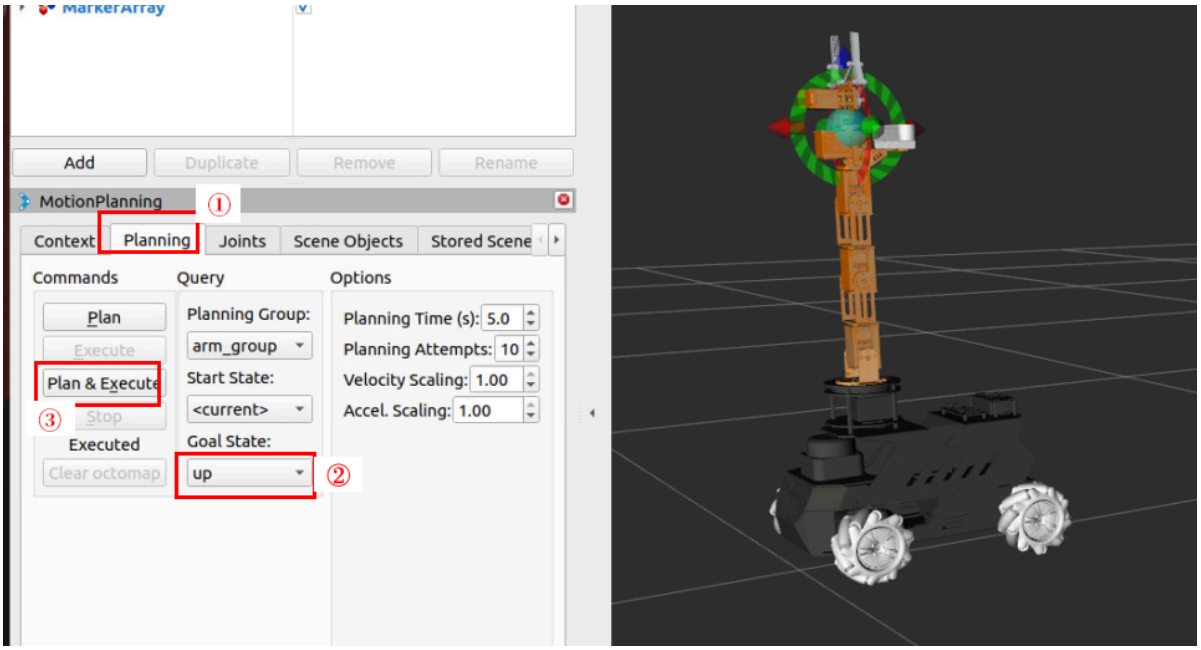
```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
```

> **Important**
>
> - In the subsequent MoveIt chapter courses, if you do not need the real machine to follow the simulated motion, just remove the `sim2real:=true` launch parameter.

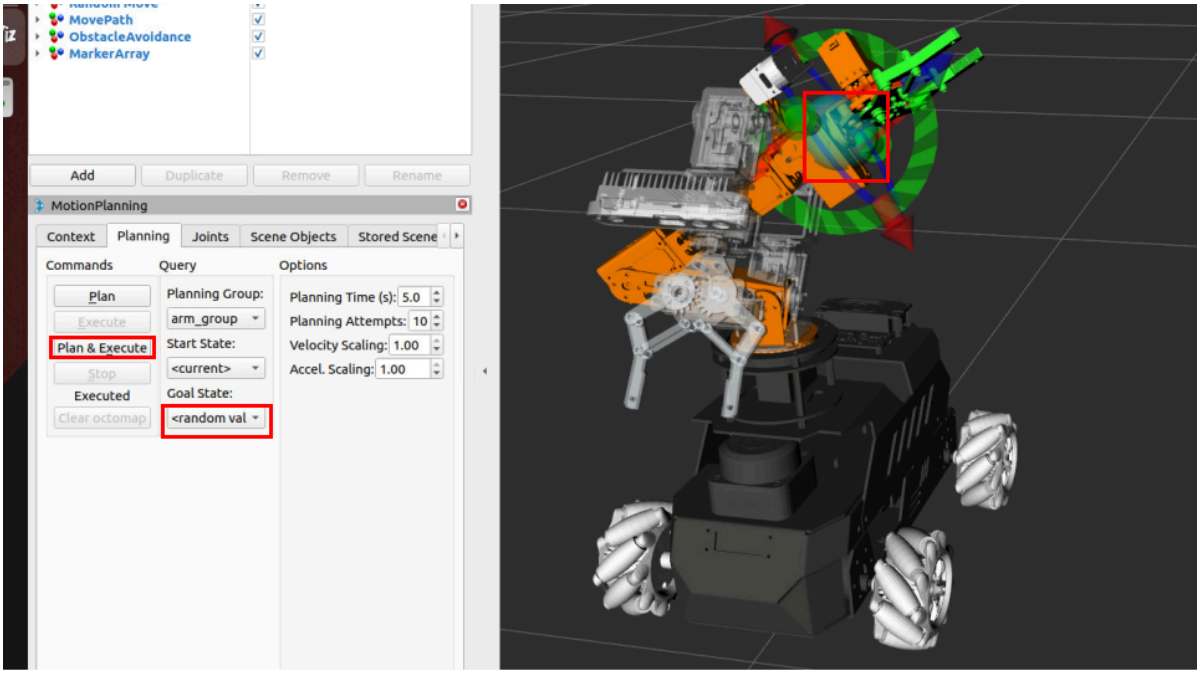
Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:



- Plan the robot arm in RViz to move to our preset init pose, as shown in the figure below. Set 【Planning Group】 to arm_group, 【Start State】 to current, and 【Goal State】 to up, then click 【Plan&Execute】; the robot arm in RViz will first plan and then slowly move to the up pose.



- In addition, you can also change the robot arm pose by dragging the ball at the end of the model in RViz, or set the Goal State to random valid (arbitrary valid pose), then click 【Plan&Execute】 to let the robot arm perform trajectory planning and motion.

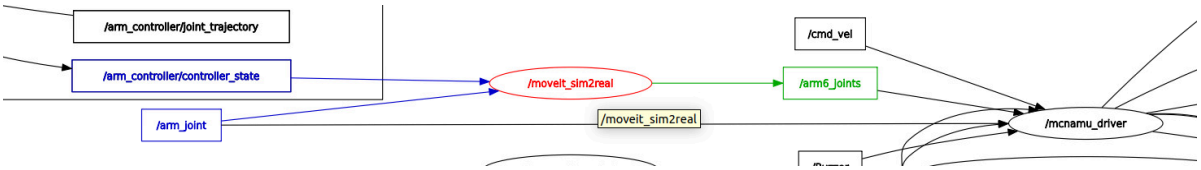


3. Node Communication

Enter the following command in the virtual machine to view the current node communication graph:

```
rqt_graph
```

Select 【Nodes/Topics(all)】 in the upper left corner, then click the refresh button next to it to get the following content. This part explains the communication among the three nodes.



4. Core Source Code Analysis

Program source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/moveit/moveit_sim2real.py
```

Initialization code description:

- `declare_parameter('init_pose', [90, 170, 0, -10, 90])`: Declares the robot arm initial pose parameter, 5 joint angle values (unit: degrees), corresponding to arm1~arm5 respectively.
- `cmd_queue = deque(maxlen=200)`: A thread-safe bounded command queue, caching up to 200 joint commands; when exceeded, the oldest command is automatically discarded.
- `_lock`: A thread lock that ensures thread safety for queue read/write operations.
- `_init_done`: A thread event flag used to start the publish thread after the initial pose setup is complete.
- `gripper_angle = 30`: The default gripper angle value (degrees), updated in real time by the gripper callback.
- `create_subscription(JointTrajectoryControllerState, "/arm_controller/controller_state")`: Subscribes to the joint trajectory state of the MoveIt2 simulation controller to obtain the real-time joint angles of the simulated robot arm.
- `create_subscription(ArmJoint, "/arm_joint")`: Subscribes to the gripper joint command topic to track the gripper opening/closing angle.
- `create_publisher(ArmJoints, "arm6_joints")`: Publishes 6-joint control commands to the real machine.
- `create_client(ArmPosition, 'set_arm_joints')`: Creates a service client for setting the robot arm joint angles.
- `_pub_thread`: A background publish thread that runs `_publish_loop` to forward the simulated joint angles to the real machine at 10 Hz.

controller_state_callback function description:

- Receives the joint state message of the MoveIt2 simulation controller, converts radians to degrees, and enqueues them.
- `zip(msg.joint_names, msg.actual.positions)`: Matches joint names with the actual angle values one by one.
- `joint_map.get('arm1', 0.0)`: Extracts the angles in the order arm1~arm5, defaulting to 0.0 when missing.

- `self.cmd_queue.append(joints_deg)` : After locking, appends the converted angle list to the command queue.

gripper_callback function description:

- Listens to the gripper command topic and updates `gripper_angle` when `msg.id == 6` (servo No. 6).

_init_arm_pose function description:

- `wait_for_service(timeout_sec=10.0)` : Waits for the `set_arm_joints` service to come online; if it times out after 10 seconds, initialization is skipped.
- `req.request_joints = list(self.init_pose)` : Sends the initial pose parameter to the real machine so that the real machine first moves to the initial pose.
- `call_async(req)` : Calls the service asynchronously; on completion, triggers `_init_arm_done_callback`.

_init_arm_done_callback function description:

- Callback after the initial pose service is complete; checks `result.success` to determine whether it succeeded.
- `finally: self._on_init_done()` : Regardless of success or failure, `_on_init_done` is eventually called to start the publish thread.

_publish_loop function description:

- Runs in a loop at 10 Hz, taking the latest joint angles from the command queue and publishing them to the real machine.
- `cmd_queue.popleft()` : After locking, pops a command from the left end of the queue; the inner while loop skips the backlogged data in between and always takes the latest value.
- `pubSixArm(joints, runtime=300)` : Publishes the angle values to the real machine; `runtime=300` means the execution time is 300 ms.

```
        self.declare_parameter('init_pose', [90, 170, 0, -10, 90])
        self.init_pose =
self.get_parameter('init_pose').get_parameter_value().integer_array_value
        self.get_logger().info(f"Init pose: {list(self.init_pose)}")

        # Thread-safe command queue (FIFO, bounded)
        self.cmd_queue = deque(maxlen=200)
        self._lock = threading.Lock()
        self._init_done = threading.Event()

        self.gripper_angle = 30

        self.sub_state = self.create_subscription(
            JointTrajectoryControllerState,
            "/arm_controller/controller_state",
            self.controller_state_callback,
            50)

        self.sub_gripper = self.create_subscription(
```

```
            ArmJoint,
            "/arm_joint",
            self.gripper_callback,
            10)

        self.SixJointPub = self.create_publisher(ArmJoints, "arm6_joints", 10)
        self.set_arm_client = self.create_client(ArmPosition, 'set_arm_joints')
        self._stop_event = threading.Event()
        self._pub_thread = threading.Thread(target=self._publish_loop,
daemon=True)
        self._init_arm_pose()

    def controller_state_callback(self, msg: JointTrajectoryControllerState):
        """Receive controller state (rad), convert to deg, enqueue."""
        if not msg.desired.positions or not msg.joint_names:
            return
        # Map joint names → degrees
        joint_map = {}
        for name, pos in zip(msg.joint_names, msg.actual.positions):
            joint_map[name] = degrees(pos)

        # Ordered arm1 ~ arm5 (matches the simulated robot)
        joints_deg = [
            joint_map.get('arm1', 0.0),
            joint_map.get('arm2', 0.0),
            joint_map.get('arm3', 0.0),
            joint_map.get('arm4', 0.0),
            joint_map.get('arm5', 0.0),
        ]

        with self._lock:
            self.cmd_queue.append(joints_deg)

    def gripper_callback(self, msg: ArmJoint):
        """Track gripper (servo 6) commands from /arm_joint topic."""
        if msg.id == 6:
            self.gripper_angle = int(msg.joint)

    def _init_arm_pose(self):
        """Call set_arm_joints service with init_pose, then start publish
thread."""
        if not self.set_arm_client.wait_for_service(timeout_sec=10.0):
            self.get_logger().warn("Failed to connect to set_arm_joints server,
skipping init arm pose")
            self._on_init_done()
            return

        req = ArmPosition.Request()
        req.request_joints = list(self.init_pose)
        self.get_logger().info(f"Calling set_arm_joints with:
{list(req.request_joints)}")
        future = self.set_arm_client.call_async(req)
        future.add_done_callback(self._init_arm_done_callback)

    def _init_arm_done_callback(self, future):
        """Callback after init pose service completes, then start bridge."""
```

```
        try:
            result = future.result()
            if result.success:
                self.get_logger().info(
                    f"Init arm pose done: {list(result.result_joints)}, msg:
{result.message}")
            else:
                self.get_logger().warn(
                    f"Init arm pose failed: {list(result.result_joints)}, msg:
{result.message}")
        except Exception as e:
            self.get_logger().error(f"Init arm pose exception: {e}")
        finally:
            self._on_init_done()

    def _on_init_done(self):
        """Start the sim2real publish bridge after init pose is complete."""
        self._init_done.set()
        self._pub_thread.start()
        self.get_logger().info("Sim2real bridge started")

    def _publish_loop(self):
        """While-loop at 10 Hz: pop latest command and publish."""
        rate = 10.0
        period = 1.0 / rate
        while not self._stop_event.is_set():
            joints = None
            with self._lock:
                if self.cmd_queue:
                    joints = self.cmd_queue.popleft()
                while self.cmd_queue:
                    joints = self.cmd_queue.popleft()
            if joints is not None:
                self.pubSixArm(joints, runtime=300)
            self._stop_event.wait(period)

    def pubSixArm(self, joints, runtime=200):
        """Convert joint degrees → ArmJoints and publish to /arm6_joints."""
        arm_joints = ArmJoints()
        arm_joints.joint1 = round(joints[0])
        arm_joints.joint2 = round(joints[1])
        arm_joints.joint3 = round(joints[2])
        arm_joints.joint4 = round(joints[3])
        arm_joints.joint5 = round(joints[4])
        arm_joints.joint6 = self.gripper_angle
        arm_joints.time = runtime
        self.SixJointPub.publish(arm_joints)

    def destroy_node(self):
        self._stop_event.set()
        if self._pub_thread.is_alive():
            self._pub_thread.join(timeout=1.0)
        super().destroy_node()
```